

Azure Cheat Sheet

Baseline: Azure CLI 2.90.0, Bicep 0.47.16, azd 1.34.2 (as of 2026-09-27). Quick reference only — verify against the official docs before production use.

CHOOSING A COMPUTE SERVICE

Service	Best for	Scaling	Scale-to-zero	State	Notes
Container Apps	Microservices, event-driven apps, Dapr	KEDA-based, per-revision	Yes	Stateless	Serverless containers on managed Kubernetes/Envoy
Container Apps Express	Small internal APIs from source, minimal config	Auto	Yes	Stateless	Fastest path via az containerapp up
App Service	Web apps/APIs, standard runtimes	Manual/auto-scale rules	Only Free/Consumption-like tiers	Stateless	Deployment slots, easy custom domains/TLS
Functions	Event-driven, short-lived executions	Auto (Consumption/Flex)	Yes (Consumption/Flex)	Stateless	Triggers/bindings; Flex Consumption for VNet+scale
AKS	Full Kubernetes control, complex topologies	Cluster/node autoscaler, KEDA	With node autoscaler to 0 (user pools)	Either	Automatic SKU reduces ops; use for advanced networking/CRDs
ACI	Single containers, burst/batch jobs	None (per-instance)	N/A (ephemeral)	Stateless	Fastest cold start for one-off container runs
VMS	Legacy/custom OS, licensing constraints	VM Scale Sets	No	Stateful	Full control, most operational overhead

MANAGED DATABASES

Engine	Azure option	Create (essentials)
SQL Server	Azure SQL Database / Managed Instance	az sql server create + az sql db create -s srv -n db --service-objective S0
PostgreSQL	Flexible Server	az postgres flexible-server create -n pg1 -g rg --sku-name Standard_D2ds_v5 --tier GeneralPurpose
MySQL	Flexible Server	az mysql flexible-server create -n mysql1 -g rg --sku-name Standard_B1ms
MongoDB / NoSQL / Cassandra / Gremlin / Table	Cosmos DB (or Azure DocumentDB for pure Mongo-compatible)	az cosmosdb create -n cdb1 -g rg --kind GlobalDocumentDB --locations regionName=westeurope
Redis	Azure Managed Redis	az redisenterprise create -n redis1 -g rg --sku Balanced_B0 --location westeurope

Entra auth: --enable-active-directory-auth (Azure SQL), Cosmos DB local-auth-disable + RBAC data-plane roles, Entra admin flags on flexible-server PostgreSQL/MySQL replace SQL--only logs for keyless auth.

HIERARCHY & REGIONS

Tenant (Entra ID) → **management group** → **subscription** → **resource group** → **resource**. Policies/RBAC assigned at any level, inherited downward.

Region: a set of datacenters with low-latency network. **Availability zone:** 1+ physically separate datacenters within a region (min. 3 zones per enabled region).

Region pair: paired region for DR, usually same geography.

Use zone-redundant SKUs (LB, Storage ZRS, zone-redundant AKS node pools) for resilience within a region; paired/secondary regions for region-wide outages.

CLTI ESSENTIALS

```
az login
az login --service-principal -u <id> -p <secret> --tenant <tid>
az account list -o table
az account set --subscription <name-or-id>
az group create -n rg-demo -l westeurope
az group delete -n rg-demo --yes --no-wait
```

Output/query: -o table|json|tsv|yaml; --query "[{.name:name}]" (JMESPath).

Extensions: az extension add -n containerapp, az extension list, az extension update -n <name>; az upgrade to update the CLI itself.

laC: az deployment group create -g rg -f main.bicep -p env=dev; az bicep build; azd up / azd deploy for azd templates.

CONTAINER APPS

```
az containerapp up -n app --source . \
  -g rg -l westeurope --ingress external
az containerapp create -n app -g rg \
  --environment env --image repo/img:tag \
  --target-port 8080 --ingress external \
  --min-replicas 0 --max-replicas 10
az containerapp update -n app -g rg \
  --image repo/img:tag2 --set-env-vars K=V
az containerapp revision list -n app -g rg
az containerapp ingress traffic set -n app -g rg \
  --revision-weight rev1=80 rev2=20
```

Scale rules: HTTP concurrency, CPU/memory, KEDA scalers (queue length, Kafka lag, cron).

Secrets: az containerapp secret set -n app -g rg --secrets key=value; reference with secretref:key.

Jobs: az containerapp job create --trigger-type Schedule|Event|Manual. Dapr enabled per app with --enable-dapr --dapr-app-id id.

ACR & AKS

```
az acr create -n myacr -g rg --sku Basic
az acr build -r myacr -t app:{{.Run.ID}} .
az aks create -n aks1 -g rg --sku automatic
az aks get-credentials -n aks1 -g rg
az aks nodepool add -g rg --cluster-name aks1 \
  -n userpool --node-count 3
az aks update -g rg -n aks1 --attach-acr myacr
az aks update -g rg -n aks1 --enable-keda \
  --enable-workload-identity --enable-oidc-issuer
az aks get-credentials -g rg -n aks1 -k <version>
```

Gateway API: enable via --enable-app-routing /AKS-managed NGINX or install the Gateway API CRDs + a supported controller (e.g. Application Gateway for Controllers).

Automatic SKU: Microsoft manages node pools, upgrades, security defaults.

KEY VAULT

```
az keyvault create -n kv1 -g rg \
  --enable-rbac-authorization true
az keyvault secret set --vault-name kv1 \
  -n MySecret --value "s3cr3t"
az keyvault secret show --vault-name kv1 -n MySecret
az role assignment create --role \
  "Key Vault Secrets User" --assignee <principal> \
  --scope <kv-resource-id>
```

App Service ref: @Microsoft.KeyVault(SecretUri=https://kv1.vault.azure.net/secrets/MySecret/).

Container Apps: managed identity + secretref backed by a Key Vault secret via --secrets name=keyvaultref:<uri>, identityref:<id>.

CSI driver (AKS): SecretProviderClass mounts secrets as files/synced K8s Secrets.

APP CONFIGURATION

```
az appconfig create -n appcfg1 -g rg --sku Standard
az appconfig kv set -n appcfg1 --key Feature:Timeout \
  --value 30 --label prod
az appconfig feature set -n appcfg1 \
  --feature Beta --yes
```

Key Vault reference: store a reference value pointing at a secret URI; App Configuration resolves it via managed identity at read time.

Kubernetes provider: AzureAppConfigurationProvider CRD syncs kv/feature flags into a ConfigMap /Secret for pods.

GATEWAYS & APIM

Decision: L4 only, cheapest → **Load Balancer** (Standard SKU), L7 HTTP(S), WAF, path routing, single region → **Application Gateway** (v2). Global L7, CDN+WAF → **Front Door**. Full API lifecycle/policies/keys → **APIM**. K8s-native L7 → **Gateway API**.

APIM tiers: Consumption (serverless), Basic v2/Standard v2 (dev/test to prod), Premium (VNet, multi-region, self-hosted gateway).

Key policies: rate-limit-by-key, validate-jwt, set-backend-service, cache-lookup / cache-store, cors.

VPN & HYBRID

```
az network vnet-gateway create -n vgw1 -g rg \
  --vnet vnet1 --gateway-type Vpn \
  --sku VpnGw2 --vpn-type RouteBased
az network vpn-connection create -n conn1 -g rg \
  --vnet-gateway1 vgw1 \
  --local-gateway2 lgw1 --shared-key <key>
az network vpn-client generate -g rg -n vgw1
```

S2S: VNet gateway + local network gateway (on-prem public IP + prefixes) + shared key/IPsec policy.

P2S with Entra ID: gateway --vpn-authentication-type AAD, plus tenant/audience/issuer config; clients sign in with Entra credentials.

EMAIL (ACS)

```
az communication create -n acs1 -g rg \
  --data-location europe
az communication email create -n email1 -g rg \
  --data-location europe
az communication email domain create -g rg \
  --email-service-name email1 -n contoso.com \
  --domain-management CustomerManaged
az communication email domain sender-username create \
  -g rg --email-service-name email1 \
  --domain-name contoso.com -n no-reply
```

Connect a verified domain to the ACS resource, then send via the **Email SDK/REST API** or **SMTP relay** (host smtp.azurecomm.net, port 587, username = ACS resource + sender, auth via Entra/SMTP username=password or Microsoft 365 relay). Quotas apply per domain/tenant tier (messages/day, recipients/message) — check current limits before high-volume sends.

MESSAGING & INTEGRATION

Service Bus: queues/topics, sessions, dead-lettering, transactions — enterprise messaging.

Event Grid: pub/sub eventing, push delivery, system + custom topics.

Event Hubs: high-throughput event streaming (Kafka-compatible).

Logic Apps: low-code workflow orchestration/connectors.

```
az servicebus namespace create -n sb1 -g rg --sku Standard
az servicebus queue create -n q1 --namespace-name sb1 -g rg
az eventgrid topic create -n topic1 -g rg
```

MONITORING

```
requests
| where timestamp > ago(1h)
| summarize count() by resultCode
exceptions
| where timestamp > ago(24h)
| top 20 by timestamp desc
AppTraces
| where SeverityLevel >= 3
```

OpenTelemetry: instrument with the OTel SDK/auto-instrumentation and export to Azure Monitor via the azuremonitor exporter (or an OTel Collector) — replaces the legacy Application Insights SDKs.

Alerts on metric/log queries; Action Groups for notification/automation.

GOVERNANCE & COST

```
az policy assignment create -n pol1 \
  --policy <policy-id> --scope <scope>
az lock create -n lock1 -g rg --lock-type CanNotDelete
az consumption budget create -n bud1 \
  --amount 500 --time-grain Monthly \
  --category Cost --start-date <date> --end-date <date>
```

Policy: enforce/audit/deny at any scope; initiatives group related policies. **Locks:** ReadOnly/CanNotDelete, apply above resource level.

Cost: budgets + alerts, Cost Analysis views, reservations/savings plans for committed use discounts.

RETIREMENTS & RENAMES — CHECK DATES BEFORE RELYING ON THEM

- Basic Load Balancer & Basic Public IP:** retired 2025-09-30 — migrate to Standard SKU.
- Application Gateway v1:** retirement targeted 2026-04-28 — migrate to v2.
- Azure Cache for Redis:** being superseded by Azure Managed Redis; legacy tiers on a multi-year retirement path through 2027/2028 — plan migration to Managed Redis.
- Azure Blueprints:** phased retirement from 2026-07-31, full retirement 2027-01-31 — migrate to Template Specs + Policy/initiatives.
- Azure Functions Linux Consumption plan:** retirement targeted 2028-09-30 — evaluate Flex Consumption.

Irueta Docs — Azure section quick reference. See the full pages for explanations, options and examples.